

Marketplace-as-Code

Subject :

Replacing manual cloud-infrastructure changes with a reviewed, version-controlled pipeline

Field : Cloud / DevOps Engineering

Team :

- Ayoub EL AZZOUZI
- Yasser BAOUZIL

Supervisor :

- Radouane BOUKHARFANE
- Hamza HERTA

Git

YAML = source of truth



CI/CD pipeline

validate · deploy · drift



Morpheus / ACS

infrastructure applied



01

The problem

How cloud infrastructure gets managed today — and why it doesn't scale.

THE PROBLEM

Infrastructure built by clicking

Teams run Morpheus through its GUI. They build it in **DEV**, then click it all again in **TEST**, then again in **PROD**.

01

No history

Nobody can say who changed what, or when.

02

No review

A wrong click in production has no safety net.

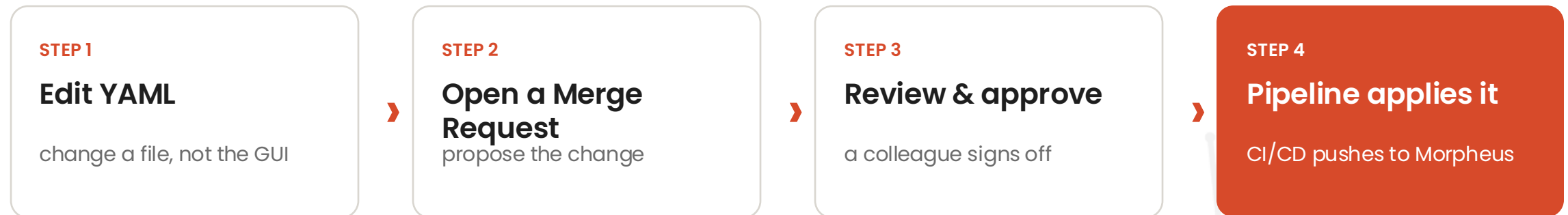
03

No rollback

When a change breaks something, there is no clean way back.

THE SOLUTION

Git becomes the source of truth



WHAT CHANGES

From clicking to a pipeline

Manual GUI — today

- Build it by hand in every environment
- Change is invisible: no log, no diff
- Promotion = redo the clicks, hope it matches
- Recovery = remember what you did

ATLAS — GitOps

- › Describe it once as YAML in Git
- › Every change is a reviewed Merge Request
- › Promotion = merge the same file forward
- › Rollback = deploy the previous commit



02

The hard part

Why one exported file cannot simply be copied between environments.

THE HARD PART

The same thing has a different number

```
blueprint.json

# raw export from DEV
"cloudId": 3,
"network": { "id": "network-17" }

# the SAME cluster in PROD
"cloudId": 1,
"network": { "id": "network-42" }
```

Inside Morpheus, clouds and networks are referenced by internal ID numbers — not names.

- Those numbers differ in every environment
- Copy a DEV export straight into PROD and it points at the wrong cloud — or fails
- This is the single hardest problem in the project

THE HARD PART

A two-step translation

 export.py

```
# EXPORT Morpheus → Git  
cloudId: 3 → cloudName:  
  "DEV-Nutanix"
```

 importer.py

```
# IMPORT Git → Morpheus  
cloudName → mapping_test.yml → id
```

- **Export replaces numbers with stable names**
cloudId 3 becomes cloudName: DEV-Nutanix
- **Import re-maps names back per environment**
a small mapping_<env>.yml file does the lookup

Files in Git are **environment-agnostic** — they describe **what** you want, not **where** it lives.



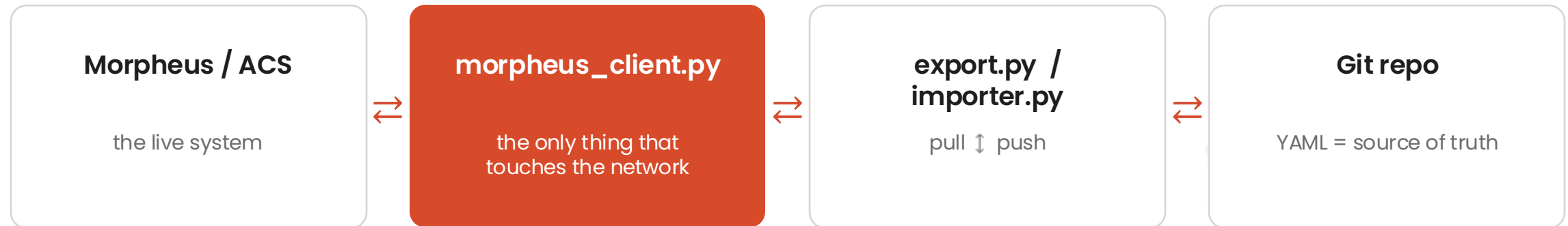
03

Architecture

Four moving parts — and one decision that makes everything else flexible.

ARCHITECTURE

One client, two directions, one watchdog



Every script talks to Morpheus through **one module**.

ARCHITECTURE

The pull, and the watchdog


export.py**Morpheus → Git, normalised**

- Strip env-specific fields: id, owner, dateCreated
- cloudId → cloudName, network.id → network.name
- Add _logical_name so the importer can resolve plans

drift_detect.py**Did anyone change Morpheus directly?**

- Export the live state the same way
- Diff against the YAML in Git with deepdiff
- Any difference fails the CI job and alerts

ARCHITECTURE

Import is idempotent and safe

- **Re-maps logical names → real IDs**
reads config/mapping_<env>.yml for the target
- **Upsert, not blind create**
update if the object exists, else create it
- **Run it as many times as you like**
matching by name keeps it idempotent

```
● ● ● preview first  
  
$ importer.py --env test --dry-run  
  validates mapping, changes nothing
```

Missing mapping = hard stop

If a name isn't in the mapping file, the importer refuses and names the missing entry — instead of silently deploying to the wrong cloud.

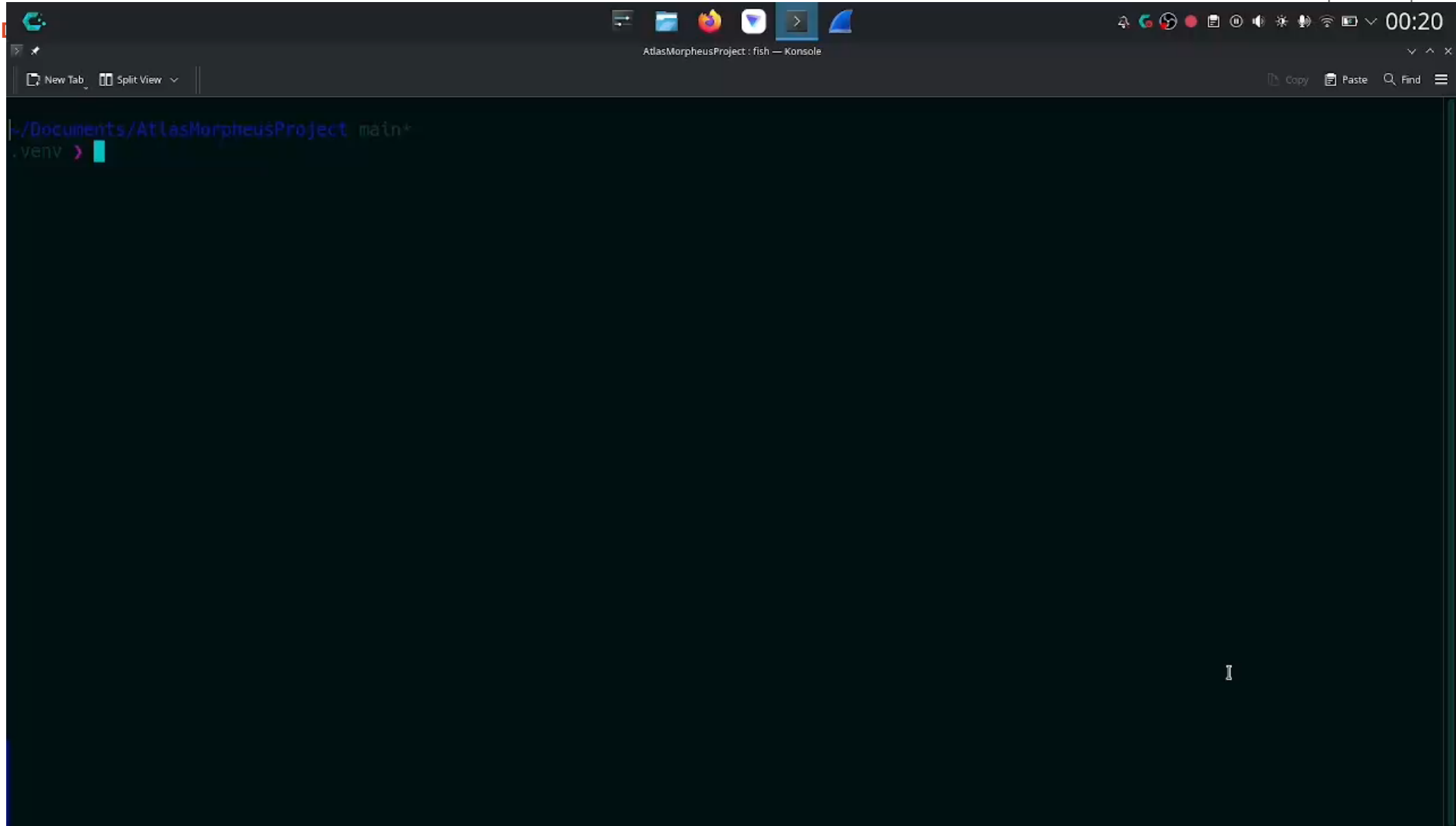


College of
Computing

04

Live demo

The full GitOps loop, end to end, on the mock — no credentials needed.



A terminal window titled "AtlasMorpheusProject : fish — Konsole" is shown. The window has a dark background and a light blue cursor. The prompt is "~ /Documents/AtlasMorpheusProject main* .venv >". The window's title bar includes standard Linux window controls and a system tray on the right with icons for network, volume, and battery. The top of the window shows a dock with various application icons.

```
~ /Documents/AtlasMorpheusProject main*  
.venv > |
```

AUTOMATION

The pipeline and its safety gate



Promotion to production needs a human click — the deliberate safety gate.

Rollback is just “deploy the previous commit”. Git is the source of truth.

STATUS

Tested, working, extensible

- **Automated test suite passes**
export transform · ID re-mapping · mock endpoints
- **End-to-end loop works on the mock**
export → review → import → drift
- **Design is deliberately extensible**
new object types plug into the same flow

Object types

Blueprints	implemented
Workflows	implemented
Policies	roadmap
Instance types	roadmap
Tasks	roadmap
Catalog items	roadmap



College of
Computing

IN ONE LINE

Manual, unreviewed, unrecoverable
becomes reviewed, versioned, automated.

Thank you